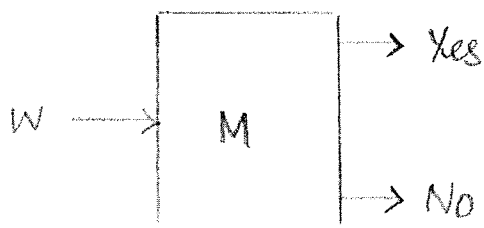


UNIT V
UNDECIDABILITY

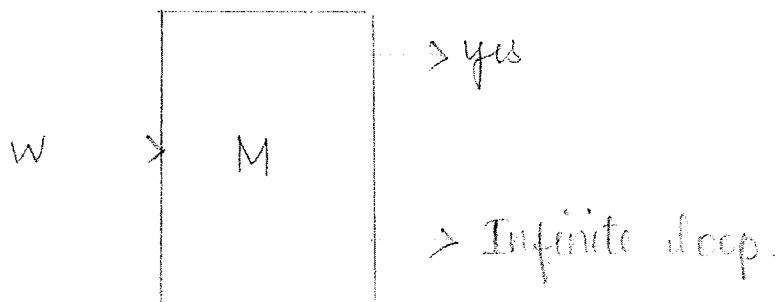
⇒ RECURSIVE LANGUAGE : (2m)

A language is recursive if there exists a Turing machine that accepts every string of the language and rejects the string that is not in the language.



⇒ LANGUAGE THAT IS NOT RECURSIVELY ENUMERABLE : (2m)

A language is recursively enumerable if there exists a Turing Machine that accepts every string of the language and does not accept strings that are not in the language.



⇒ DIAGONALIZATION LANGUAGE (L_d) : (2m)

The diagonalization language L_d , is the set of strings w_i where w_i is not in $L(M_i)$.

$\therefore L(M_i) = \emptyset$ if w_i valid TM code.

L_d consists of all strings w such that the TM 'M' whose code in w does not accept the input w .

	$j \rightarrow$	1	2	3	4	...
$i \downarrow$	1	0	1	0	0	...
	2	1	0	1	1	...
	3	1	1	0	0	...
	4	1				...
	...					...

Diagonal.

It is clear that whether the $T_m M_i$ accepts the input string or not.

If $(i, j) = 1$, Yes it is accepted.

If $(i, j) = 0$, No it is rejected.

⇒ DIAGONALIZATION : (2m)

The process of complementing the diagonal to construct the characteristic vector of a language that cannot be the language that appears in any row is called Diagonalization.

Then the complement of the diagonal cannot be the characteristic vector of any Turing machine.

THEOREM 1 : L_d is not recursively enumerable

L_d is not a recursively enumerable language.

(ie) There is no Turing Machine that accepts L_d .

PROOF: Suppose L_d is accepted by some TM M defined by $L(M)$. Since L_d is a language over alphabet $\{0, 1\}$, M would be in the list of Turing Machines constructed, where it includes all Turing machines with input alphabet $\{0, 1\}$. So there may be atleast one code for M , say i that is

$$M = M_i$$

Then

By definition, $L_d = \{w_i \mid M_i \text{ does not accept } w_i\}$

Here we have two possibilities

- $w_i \in L_d$

* This means that (i, i) entry is '0' and so M_i does not accept w_i . But our assumption there is that there exists a Turing machine M_i , which accepts w_i . There is a contradiction.

- $w_i \notin L_d$

* This means that (i, i) entry is '1' and so M_i accept w_i . But by definition of L_d , M_i does not accept w_i . So there is a contradiction.

Thus it is clear that L_d is not recursively enumerable, and L_d is not recursive too.

COMPLEMENTATION OF RECURSIVE AND RECURSIVELY ENUMERABLE LANGUAGES

The recursive languages are closed under complementation.

THEOREM : 2 If L is a recursive language, so is $\bar{L}$

Let L be a recursive language and M a Turing Machine that halts on all inputs and accepts L .

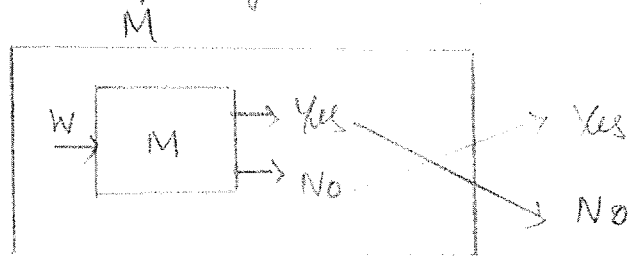
Construct a Turing Machine M' from M such that,

(i) The accepting states of M are made non-accepting states of M' with no transitions.

(ii) Create a new accepting state P which has no transitions.

(iii) For each pair of non-accepting state of M and a tape symbol of M such that M has no transition. Make the transition to the accepting state P .

If M enters a final state on input w , then M' halts without accepting.



If M halts without accepting, M' enters to final state.

M
⇒ Accept → final state
⇒ Rejects

M'
Rejects.
Accept → final state.

It is clear that M' has either of the event accepts or rejects.

So M' accepts I which is also recursive.

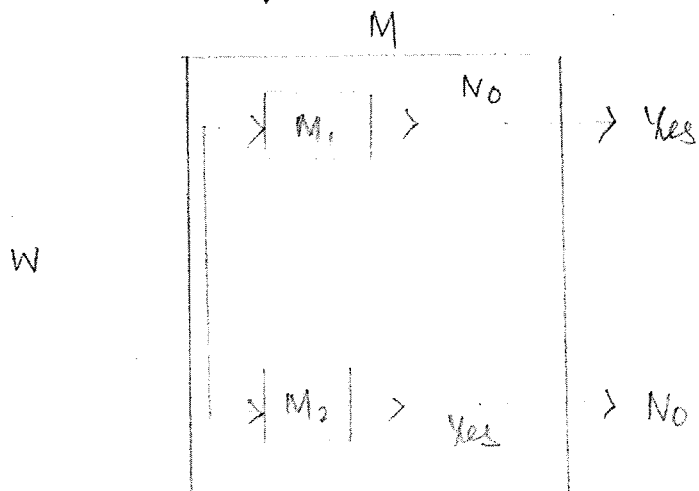
THEOREM 3: If a language L and its complement $\bar{L}$ are both recursively enumerable, then L is recursive.

Proof: Let M_1 and M_2 be the Turing Machine which accepts the language L and $\bar{L}$ respectively.

construct M to simulate simultaneously M_1 and M_2 .

M accepts w if M_1 accepts w .

M ~~rejects~~ ^{rejects} w if M_2 accepts w .



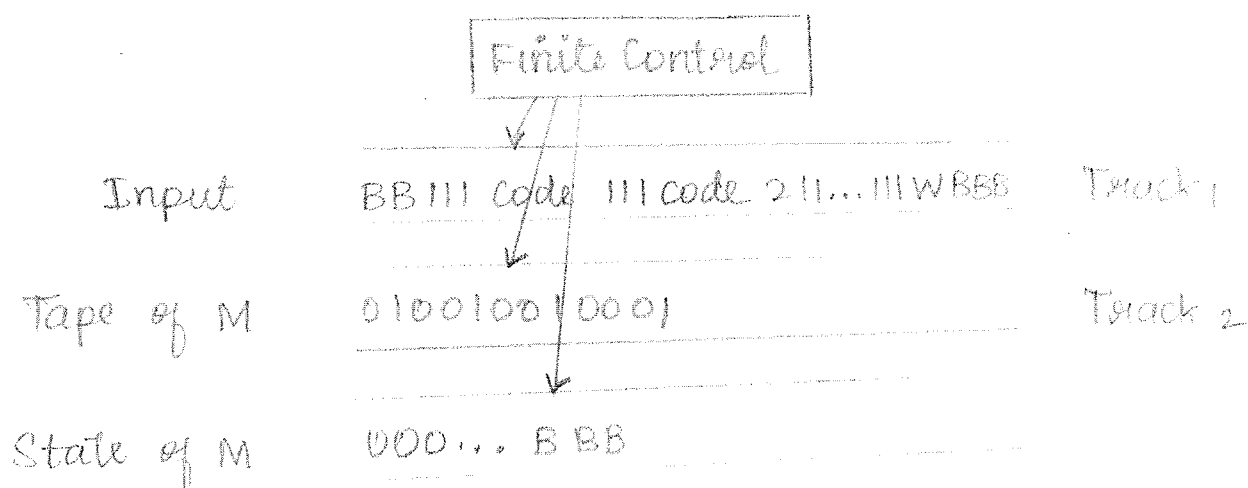
w is in either L or $\bar{L}$, so exactly one of M_1 or M_2 , will accept.

Obviously M will always say either 'Yes' or 'No' but not both.

THEOREM 4: L_u is recursively enumerable.

PROOF: In order to prove this theorem, it is necessary to construct a Turing machine that accepts L_u . The Turing machine U consists of a three

track input tape where the first track holds the input tape $\langle M, w \rangle$, the second track contains the tape of M where tape symbols are written in unary form and the third track represents the state q_i which is also in unary form.



The operation of U are as follows:

- 1.) First make sure that the code for M is a legitimate code for some Turing machine M . Otherwise it halts without accepting.
- 2.) Initialize the second tape with the input w , in its encoded form. keep o the start state of M on the third tape and move the head of U 's second tape of the first stimulated cell.
- 3.) If o^i is the current state with o^i the current input symbol appeared on track three and two respectively then U finds the corresponding transition of the form $o; i o_j i o_k i o_l i o_m$ on track 1 and replace o^i by o^k and o^j by o^l .

4) Move the head on tape two to the position corresponding to the value of M .

5) The universal Turing Machine U accepts $0^i 1 0^j 1 0^k 1 0^l 1 0^m$ if M has the transition of the form $\delta(0^i, 0^j) = (0^k, 0^l, 0^m)$. Otherwise M halts without accepting.

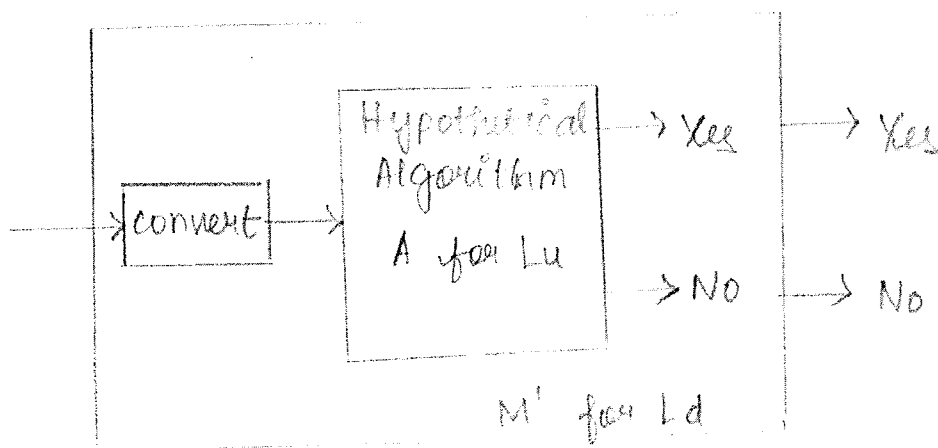
Thus U simulates M and accepts w . Thus L_U is recursively enumerable.

THEOREM 5: L_U is Recursively enumerable but not recursive.

PROOF :

It is already proved that L_U is RE.

Assume L_U is recursive. According to the closure property of recursive sets, the complement of L_U (i.e.) $\bar{L}_U$ is also recursive.



Suppose A is an algorithm recognizing L_U . L_d can be recognized as follows:

⇒ Given string w in $(0+1)^*$ determined by an easy calculation, the value of i such that $w = w_i$. Integer i in binary is the corresponding code for

for some TMM_i .

→ Give the input $\langle M_i, w_i \rangle$ to algorithm A and accept w if and only if M_i accepts w_i .

→ So the constructed algorithm also accepts w if and only if $w = w_i$ which is in $L(\overline{M_i})$. This is the algorithm constructed for L_d .

But no such algorithm exists and also our assumption of there is an algorithm A for L_u exists is false.

Hence L_u is RE but not recursive.

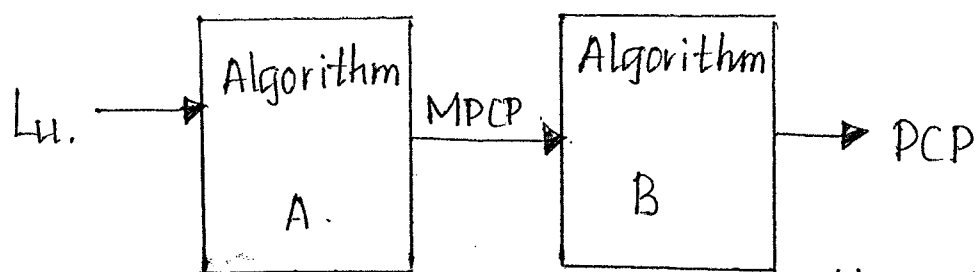
UNIVERSAL LANGUAGE (L_u) : (2m)

$$L_u = \{ \langle M, w \rangle \mid M \text{ accepts } w \}$$

L_u is the set of strings representing a TM and an input accepted by that TM. So there is a TM U called Universal Turing Machine.

POST CORRESPONDENCE PROBLEM

In this, the undecidable problem about Turing machine are reduced to undecidable problems about real things. The goal is to prove that Problem about strings to be undecidable



We reduce L_u to modified PCP then to PCP using algorithm.

Definition

An instance of Post's Correspondence problem (PCP) consists of two lists of strings over Σ

$$A = w_1, w_2, \dots, w_k.$$

$$B = x_1, x_2, \dots, x_k \text{ for some integer } k$$

The instance of PCP has a solution if there is any sequence of integers $i_1, i_2, \dots, i_m$ with $m > 1$ such that

$$w_{i_1} w_{i_2} w_{i_3} \dots w_{i_m} = x_{i_1} x_{i_2} \dots x_{i_m}.$$

is a solution to this instance of PCP

MODIFIED POST'S CORRESPONDENCE PROBLEM

[MPCP]

Modified PCP

1. In order to simplify the reduction of L_u to PCP, an intermediate version of PCP called Modified Post's Correspondence problem is used.

The Modified PCP is the following

Given lists A and B, of k string each from Σ^*

$$A = w_1, w_2, \dots, w_k \quad B = x_1, x_2, \dots, x_k.$$

It has the solution such that

$$w_{i_1} w_{i_2} \dots w_{i_r} = x_{j_1} x_{j_2} \dots x_{j_r}.$$

The difference between the MPCP and PCP is that in the MPCP, a solution is required to be stored with the first string on each list.

Theorem 6

Statement:

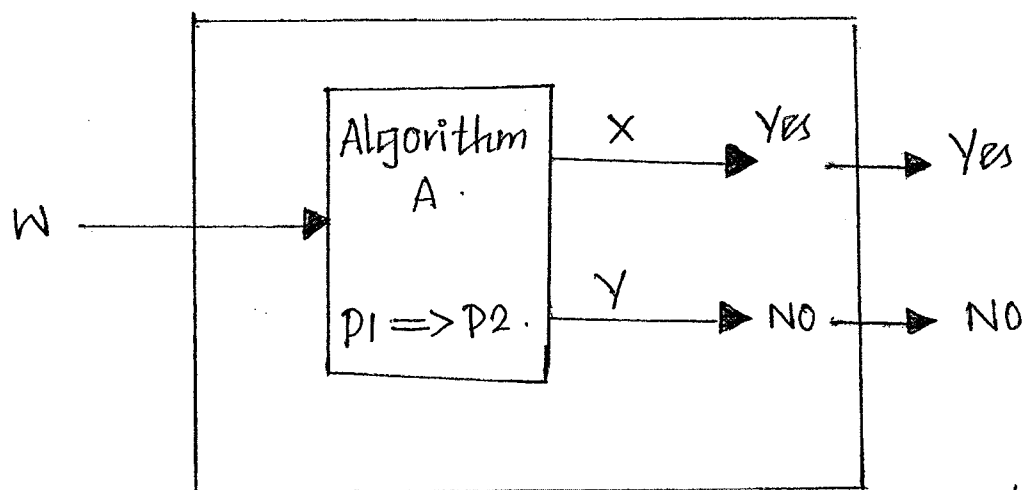
If there is a reduction from P_1 to P_2 then

i) If P_1 is undecidable, then so is P_2

ii) If P_2 is non-RE, then so is P_1

Proof:

i) If P_2 is undecidable, then so is P_1
 Assume P_1 is undecidable. Let A be the algorithm which converts the instances of P_1 to instances of P_2



Suppose w be the instances of P_1 , given to the algorithm A that converts w into an instance x of

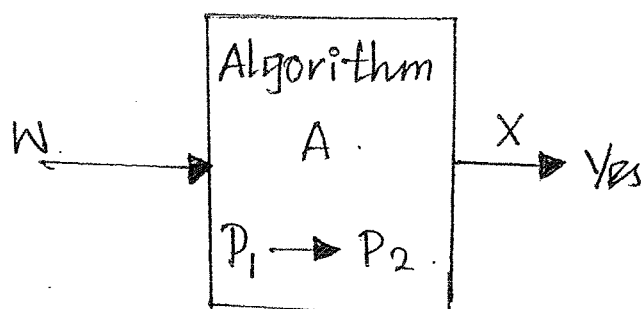
Algorithm output	Inference
Yes	x is in P_2 [$\because w$ is in P_1]
No	x is not in P_2 [$\because w$ is not in P_1]

This is the contradiction to our assumption that P_1 is undecidable

Thus, if P_1 is undecidable, then P_2 is also undecidable.

ii) If P_1 is non-RE, then so is P_2

Assume that P_1 is non-RE, but P_2 is RE. Since P_2



If w is in P_1 then x is in P_2 , so the TM will accept w

If w is not in P_1 , then x is not in P_2 , so the TM may or maynot halt but will not accept w

This is a contradiction to our assumption. Thus if P_1 is non-RE, then P_2 is non RE

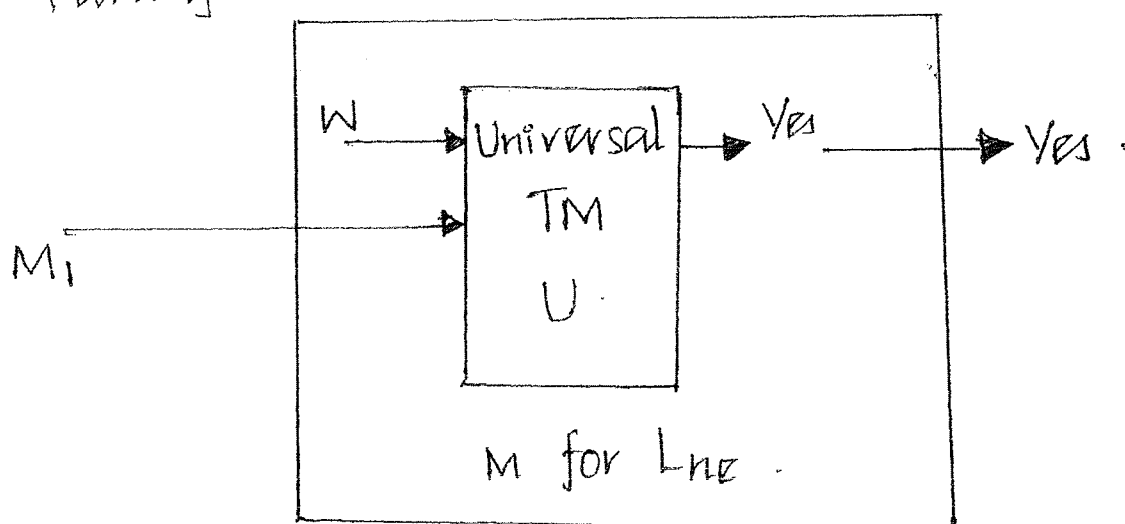
Theorem 7

Statement :

L_{ne} is recursively enumerable

Proof :

The construction is based on a non-deterministic Turing machine



(2)

The theorem is proved as follows

- i. A Turing Machine code M_i is given as input to the TM
- ii. A guesses an input w in a right way that M_i might accept.
- iii. M is simulated to the Universal Turing Machine U , which tests whether M_i accepts w .
- iv. If M_i accepts w , then M accepts w

Thus M_i accepts any string that will be guessed right by the Turing Machine M . If $L(M_i) = \emptyset$, then no guess is made by Turing Machine M , so M does not accept M_i .

$$\text{Thus } L(M) = L_{ne}$$

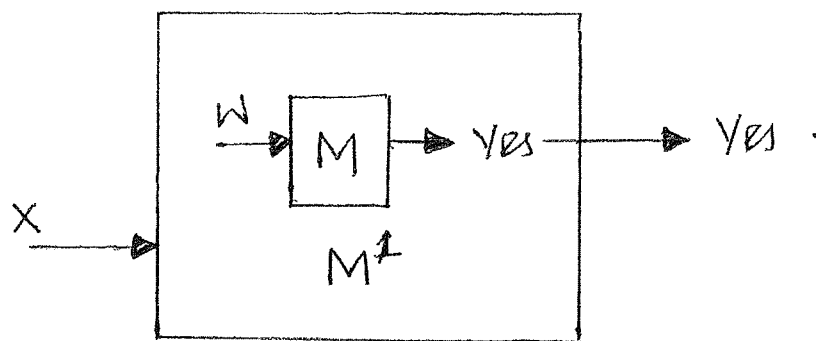
Theorem 8

Statement:

L_{ne} is not recursive.

Proof:

The algorithm of this theorem should be designed in such a way that it converts an input that is binary-coded pair (M, w) into a Turing Machine M^1 such that $L(M^1) = \emptyset$ if and only if M accepts input w



The Turing Machine M^1 is designed to perform the below operation

1. M ignores its own input x rather it replaces its input by w , the input string accepted by Turing Machine M . M is designed to accept a specific pair (M, w) whose length is n , having a sequence of n states like $q_0, q_1, q_2, \dots, q_n$, Where q_0 is the start state

- a) For $i = 0, 1, \dots, n-1$, if the turing machine is at state q_i , M writes $(i+1)$ st bit of the code for (M, w) and goes to state q_{i+1} moving right.

- b) In state q_n , M moves right by replacing any non blanks to blanks

2. When the Turing machine M reaches a blank in state q_n , it uses a similar collection of states to reposition its head at the Left end of the tape.

3. M simulates a universal Turing machine U on its present tape. If U accepts, then M accepts, if U does ^{not} accept, M never accepts. The simulation made here is by reduction of L_u to L_{ne} .

Assume L_{ne} is recursive. Then the algorithm for L_u is as follows

1. Convert (M, w) to the Turing machine M^1 by reduction of L_u to L_{ne}

2. By the hypothetical algorithm of L_{ne} , Tell

i) If $L(M^1) = \emptyset$, M does not accept w

ii) If $L(M^1) \neq \emptyset$ M accepts w

By the algorithm of L_u , this is not true. Our assumption is contradictory and conclude that L_{ne} is not recursive.

Theorem 9

Statement

MPCP reduces to PCP

Proof:

Assume that $i_1, i_2, \dots, i_m$ is a solution to the given MPCP instance with lists A and B

We know that, according to MPCP

$$W_1 W_{i1} W_{i2} \dots W_{im} = X_1 X_{i1} \dots X_{im}$$

Replace w 's by y 's and x 's by z 's. Then we have the string like $y_1, y_{i1}, y_{i2} \dots y_{im}$ and $z_1, z_{i1}, z_{i2} \dots z_{im}$. The only difference is that the first string would be missing a $*$ at the beginning and the second string would be missing a $*$ at the end

$$* y_1, y_{i1}, y_{i2} \dots y_{im} = z_1, z_{i1}, z_{i2} \dots z_{im}$$

Using the construction rule, Put $y_0 = * y_1$ and $z_0 = z_1$. Then fix the initial $*$ by replacing the first index by 0

$$y_1, y_{i1}, y_{i2} \dots y_{im} = z_1, z_{i1}, z_{i2} \dots z_{im} *$$

Append the index $k+1$ i.e. $y_{k+1} = \$$ and $z_{k+1} = *$
 $z_{k+1} = * \$$

$$* y_1, y_{i1}, y_{i2} \dots y_{im} y_{k+1} = z_1, z_{i1}, z_{i2} \dots z_{im} z_{k+1}$$

Thus $0, i_1, i_2, \dots, i_m, k+1$ is a solution to the instance of PCP.

For MPCP, $i_1, i_2, \dots, i_m$ is a solution. If we remove the $*$'s and the final $\$$ from the string $Y_1, Y_{i_1}, Y_{i_2} \dots Y_{i_m} Y_{K+1}$, we get the string

$$W_1 W_{i_1} W_{i_2} \dots W_{i_m}$$

If we remove $*$'s and $\$$ from the string $Z_1, Z_{i_1}, Z_{i_2}, \dots, Z_{i_m} Z_{K+1}$, we get

$$X_1 X_{i_1} X_{i_2} \dots X_{i_m}$$

So for PCP

$$Y_1, Y_{i_1}, Y_{i_2} \dots Y_{i_m} Y_{K+1} = Z_1, Z_{i_1}, Z_{i_2} \dots Z_{i_m} Z_{K+1}$$

Which follows that

$$W_1 W_{i_1} W_{i_2} \dots W_{i_m} = X_1 X_{i_1} X_{i_2} \dots X_{i_m}$$

Thus a solution to the PCP instance implies a solution to the MPCP instance. Thus there is a reduction of MPCP to PCP, which confirms that if PCP were decidable, PCP would also be decidable.

Theorem 10

Statement

Post's Correspondence problem is undecidable

Proof:

The proof of this theorem is telling how to reduce L_u to MPCP. It can be proved using the statement

"M accepts w if and only if the constructed MPCP instance has a solution".

If Part

MPCP instance has a solution

To Prove: M accepts w

Assume a partial solution begins with

$A : \#$

$B : \#q_0 w \#$

states and the tape symbols can only be handled by the pairs of rule (3) and all other tape symbols and $\#$ must be handled by pairs of from rule (2)

After reaching the accepting states, the rules (4) and (5) can be used. Thus, unless M reaches an accepting state, all partial solutions have the

form

$A : x$

$B : xy$

Where

x - sequence of ID's of M representing a computation of M on input w possibly followed by $\#$ and the beginning of the next ID a

y - completion of a , $\#$ and beginning of the ID that follows a , upto the point that x ended within a itself

Thus as long as M does not enter an accepting state, the partial solution is not a solution and also B is longer than A .

Only-if part M accepts w

If w is in $L(M)$, then start with the pair from rule (1) and simulate the computation of M on w .

And using the rule (3) pair to copy the state from each ID and simulate one move of M . Then by using rule (2), to copy tape symbols and the marker $\#$ as needed. If M reaches an accepting state, then the pairs from rule (4) and rule (5) allow the A string to catch upto the B string and form a solution.

Thus if there is a solution M must enter an accepting state and so M accepts w

2 mark

20. Define the classes P and NP ?

P consists of all those languages or problems accepted by some Turing Machine that runs in some polynomial amount of time, as a function of its input length Ex: Kruskal's problem.

NP is the class of language or problems that are accepted by non-deterministic TM's with a polynomial bound on the time taken along any sequence of non-deterministic choices and also Verifiable in Polynomial time.
Ex: Travelling salesman

5. Define NP hard problem?

A problem is said to be NP hard if there is a polynomial time reduction i.e. can be solvable in polynomial time.

Example:

Decision problems, search problems.

13. Define the basic recursive primitive function?

- i) Zero function $z(x) = 0$
- ii) Successor function $s(x) = x + 1$
- iii) Projection function

$$P_1(x_1, x_2) = x_1$$

$$P_2(x_1, x_2) = x_2$$

14. What do you mean by NP Complete problem?

A language L is NP Complete if it satisfies the following condition

1. L is in NP
2. For every language L' in NP there is a polynomial-time reduction of L' to L

Example: Travelling Salesman Problem.

19. What are the properties of recursively enumerable sets which are undecidable?

1. Emptiness
2. Finiteness
3. Regularity
4. Context-freeness

